

```
public class SortingAlgorithms {

    public static class SortResult {

        public int comparisons;

        public int swaps;

        public int shifts;

        public SortResult() {

            comparisons = 0;

            swaps = 0;

            shifts = 0;

        }

    }

    public static SortResult selectionSort(int[] a) {

        SortResult r = new SortResult();

        for (int i = 0; i < a.length - 1; i++) {

            int minIndex = i;

            for (int j = i + 1; j < a.length; j++) {

                r.comparisons++;

                if (a[j] < a[minIndex]) {

                    minIndex = j;

                }

            }

            if (minIndex != i) {

                int temp = a[i];

                a[i] = a[minIndex];

                a[minIndex] = temp;

            }

        }

    }

}
```

```

        r.swaps++;
    }
}

return r;
}

public static SortResult insertionSort(int[] a) {
    SortResult r = new SortResult();

    for (int i = 1; i < a.length; i++) {
        int key = a[i];
        int j = i - 1;

        while (j >= 0) {
            r.comparisons++;

            if (a[j] > key) {
                a[j + 1] = a[j];
                r.shifts++;
                j--;
            } else {
                break;
            }
        }

        a[j + 1] = key;
    }

    return r;
}

```

```
public static SortResult mergeSort(int[] a) {  
    SortResult r = new SortResult();  
    mergeSortRecursive(a, 0, a.length - 1, r);  
    return r;  
}
```

```
private static void mergeSortRecursive(int[] a, int left, int right, SortResult r) {  
    if (left >= right) {  
        return;  
    }
```

```
  
    int mid = left + (right - left) / 2;  
    mergeSortRecursive(a, left, mid, r);  
    mergeSortRecursive(a, mid + 1, right, r);  
    merge(a, left, mid, right, r);  
}
```

```
private static void merge(int[] a, int left, int mid, int right, SortResult r) {  
    int leftSize = mid - left + 1;  
    int rightSize = right - mid;  
  
    int[] L = new int[leftSize];  
    int[] R = new int[rightSize];  
  
    for (int i = 0; i < leftSize; i++) {  
        L[i] = a[left + i];  
    }  
  
    for (int j = 0; j < rightSize; j++) {  
        R[j] = a[mid + 1 + j];  
    }
```

```
int i = 0;
```

```
int j = 0;
```

```
int k = left;
```

```
while (i < leftSize && j < rightSize) {
```

```
    r.comparisons++;
```

```
    if (L[i] <= R[j]) {
```

```
        a[k++] = L[i++];
```

```
    } else {
```

```
        a[k++] = R[j++];
```

```
    }
```

```
}
```

```
while (i < leftSize) {
```

```
    a[k++] = L[i++];
```

```
}
```

```
while (j < rightSize) {
```

```
    a[k++] = R[j++];
```

```
}
```

```
}
```

```
public static SortResult quickSort(int[] a) {
```

```
    SortResult r = new SortResult();
```

```
    quickSortRecursive(a, 0, a.length - 1, r);
```

```
    return r;
```

```
}
```

```
private static void quickSortRecursive(int[] a, int low, int high, SortResult r) {
```

```

    if (low < high) {
        int pivotIndex = partition(a, low, high, r);
        quickSortRecursive(a, low, pivotIndex - 1, r);
        quickSortRecursive(a, pivotIndex + 1, high, r);
    }
}

private static int partition(int[] a, int low, int high, SortResult r) {
    int pivot = a[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        r.comparisons++;

        if (a[j] <= pivot) {
            i++;
            int temp = a[i];
            a[i] = a[j];
            a[j] = temp;
            r.swaps++;
        }
    }

    int temp = a[i + 1];
    a[i + 1] = a[high];
    a[high] = temp;
    r.swaps++;

    return i + 1;
}

```

```
public static void printArray(int[] a) {  
    System.out.print("[");  
    for (int i = 0; i < a.length; i++) {  
        System.out.print(a[i]);  
        if (i < a.length - 1) {  
            System.out.print(", ");  
        }  
    }  
    System.out.println("]");  
}
```

```
public static int[] copyArray(int[] original) {  
    int[] copy = new int[original.length];  
    for (int i = 0; i < original.length; i++) {  
        copy[i] = original[i];  
    }  
    return copy;  
}
```